PROJECT REPORT — HOSPITAL MANAGEMENT SYSTEM (HMS360)

Author

Name: Syed Akash Ahmed
Roll Number: 24f3000578
Email: 24f3000578@ds.study.iitm.ac.in

About Me: I am a passionate software development student interested in backend development, database systems, and web applications. I enjoy building real-world solutions using Python and web technologies.

AI/LLM Usage

This project implementation was supported by guidance from **ChatGPT (OpenAI GPT-5.1)** to assist in writing SQLAlchemy model definitions, clarification of concepts, debugging assistance, documentation samples, and improving variable naming consistency.

The extent of AI/LLM usage is around **15–20%**, limited to **code suggestions and documentation formatting**.

All core implementation and integration was manually coded, tested, and validated by me.

Description

This project is a Hospital Management System designed to manage doctors, patients, departments, appointments, and medical treatment histories through separate dashboards for Admin, Doctor, and Patient roles. Users can book and manage appointments, update treatment records, search records, and control access based on their role.

Technologies Used

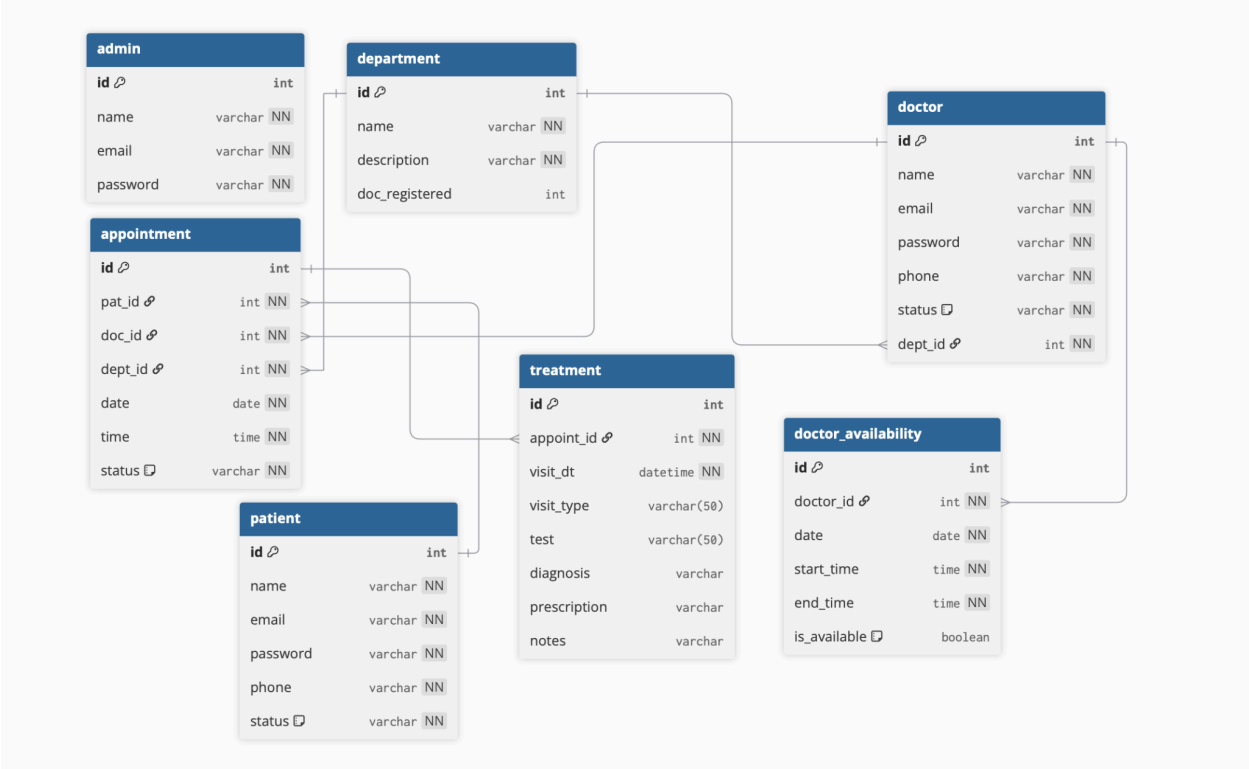
Technology	Purpose
Python & Flask	Backend application development & routing
SQLite	Database and storage
Flask-Login	Authentication and session management
Flask-SQLAlchemy	ORM for models and database operations
HTML, CSS, Bootstrap	Frontend UI and responsive styling
Jinja2	Dynamic templating engine for Flask
DBML / ER Diagram	Database design and visualization

DB Schema Design

The database consists of 7 tables:

Tables and Columns

Table	Important Columns / Constraints
Admin	id (PK), email (unique), password
Department	id (PK), name (unique), description
Doctor	id (PK), dept_id (FK), status (default: Active), email (unique)
Patient	id (PK), email (unique), phone, status (default: Active)
Appointment	id (PK), pat_id (FK), doc_id (FK), date, time, status (default: Booked)
Treatment	id (PK), appoint_id (FK), diagnosis, prescription, notes
DoctorAvailability	doctor_id (FK), date, start_time, end_time, is_available



Reasoning Behind Design

- Separate tables improve normalization and reduce duplication.
- Appointment table acts as a central link between patient, doctor, and department.
- Treatment is separated to store historical multiple visit records.
- DoctorAvailability simplifies scheduling functionality.
- Status and constraints support business logic & clean filtering.

Architecture and Features

The project follows **MVC-based structure**:

Component	Location / Description
Controllers / Routes	`app.py` and additional modular route sections
Templates / UI	`templates/` folder using Jinja2 and Bootstrap
Models / DB Entities	`models.py` using SQLAlchemy
Static Resources	Bootstrap CDN used; no static folder required for custom CSS/JS.

Features Implemented

- Role-based login system for Admin, Doctor, and Patient
- Search functionality across dashboards
- Appointment booking with duplicate-slot prevention
- Doctor availability management
- Patient treatment history maintained & editable
- Admin management for doctors, departments, and patients
- Flash alert messages for all operations
- Modal-based CRUD operations for better UX
- Cancellation & status control for appointments

Additional Features Beyond Requirements

- Blacklist & whitelist functionality for users
- Auto-prefill last treatment on update
- Dashboard statistics (total counts)
- Fully responsive UI structure

Video Demonstration

Video Link:
https://drive.google.com/file/d/1-iQJw7b9FrnWA47m44JsWpH7yKfRfhiu/view?usp=sharing